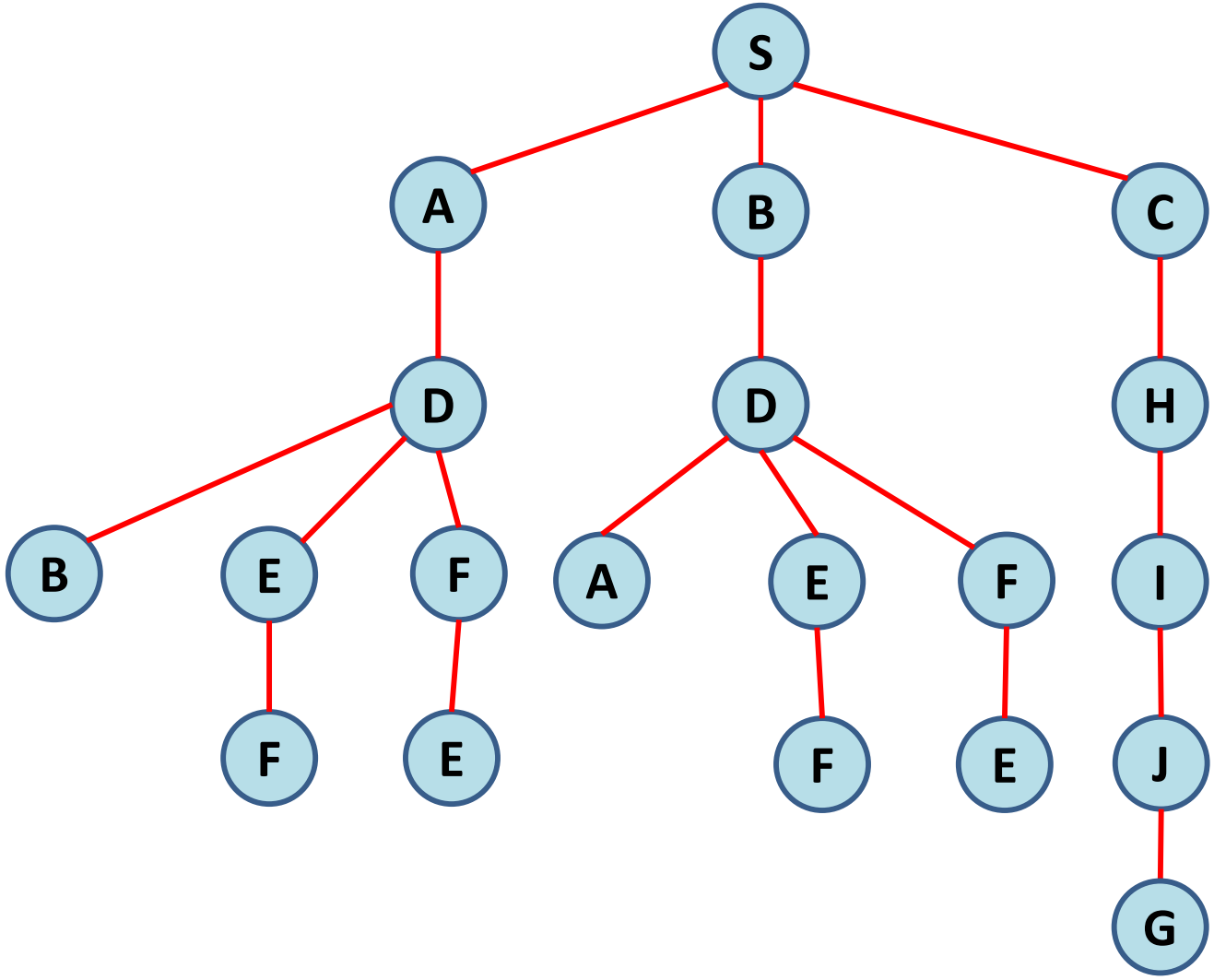
TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

Let’s apply the following algorithm to solve the problem:

Best-First Search with the Visited List

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (h-val S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q, including their h-values; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

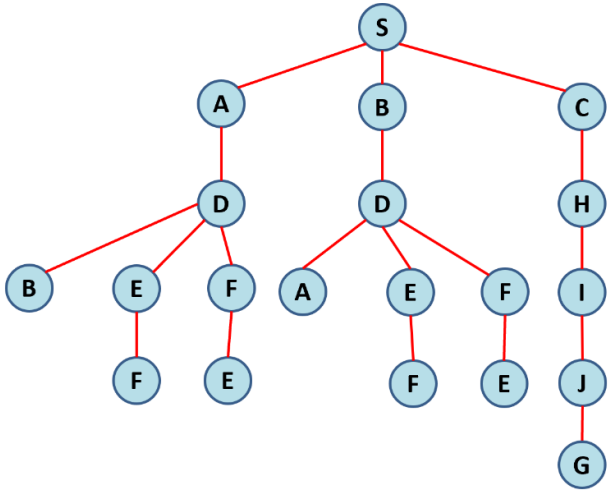
2) Best-First Search: Pick element from Q according to lowest heuristic .

6) Best-First Search : Add path extensions to **front** of Q. (our convention)

Best-First Search with Visited List

Step	Dequeued	Q	Visited
0		(10S)	S
1	(10S)	(5AS) (8BS) (6CS)	S,A,B,C

Search Tree



Heuristic values:

S = 10, A = 5, B = 8, C = 6,
D = 7, E = 6, F = 5, H = 3,
I = 2, J = 1, G = 0

Pick element from Q, based on lowest heuristic

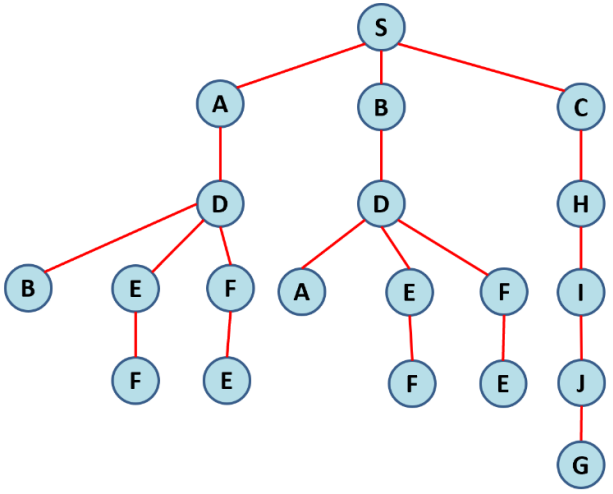
Add path extensions to **front** of Q (convention). Q is best implemented using a **priority queue**.

SEARCH

Best-First Search with Visited List

Step	Dequeued	Q	Visited
0		(10S)	S
1	(10S)	(5AS) (8BS) (6CS)	S,A,B,C
2	(5AS)	(7DAS) (8BS) (6CS)	“, D

Search Tree



“ – means including all previous states visited

Heuristic values:

S = 10, A = 5, B = 8, C = 6,
D = 7, E = 6, F = 5, H = 3,
I = 2, J = 1, G = 0

Pick element from Q, based on lowest heuristic

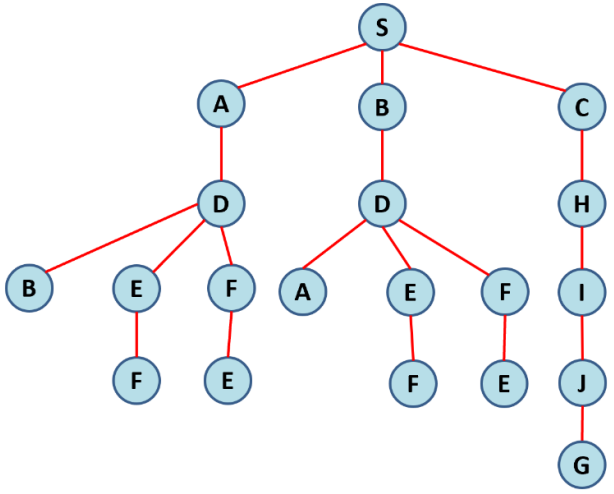
Add path extensions to **front** of Q (convention). Q is best implemented using a **priority queue**.

SEARCH

Best-First Search with Visited List

Step	Dequeued	Q	Visited
0		(10S)	S
1	(10S)	(5AS) (8BS) (6CS)	S,A,B,C
2	(5AS)	(7DAS) (8BS) (6CS)	“, D
3	(6CS)	(3HCS)(7DAS)(8BS)	“, H
4	(3HCS)	(2 IHCS) (7DAS)(8BS)	“, I
5	(2 IHCS)	(1 JIHCS) (7DAS)(8BS)	“, J
6	(1 JIHCS)	(0 GJIHCS) (7DAS)(8BS)	“, G
7	(0 GJIHCS)	(7DAS)(8BS)	

Search Tree



Heuristic values:

S = 10, A = 5, B = 8, C = 6,
D = 7, E = 6, F = 5, H = 3,
I = 2, J = 1, G = 0

Pick element from Q, based on lowest heuristic

Add path extensions to **front** of Q (convention). Q is best implemented using a **priority queue**.

Best-First Search with Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Path length = ?

Max Q Length = ?

Number of state expansions = ?

SEARCH

Best-First Search with Visited List

Characterisation:

Sequence of state expansions = S-A-C-H-I-J-G

Path = S-C-H-I-J-G

Path length = 5

Max Q Length = 3

Number of state expansions = 7

Best-First Search with the Visited List expanded the least number of states (7) among the 3 algorithms being compared. Moreover, it used memory space efficiently. This is due to the accuracy of the heuristic used. In the programming assignment, we will see some variations in the performance when the heuristic function is varied.